

Homework Report: LocalGPT Chat

FastAPI application with PostgreSQL, Alembic migrations, Redis refresh sessions, JWT access tokens, GitHub OAuth and a local LLM integration point.

1. API structure

Group	Method / route	Purpose
Auth	POST /auth/register	Create a user with unique login and hashed password.
Auth	POST /auth/login	Password login; returns JWT access token and refresh token.
Auth	POST /auth/refresh	Rotates Redis refresh session and issues a new access token.
Auth	POST /auth/logout	Deletes refresh session from Redis.
OAuth	GET /auth/github/login	Redirects user to GitHub OAuth.
OAuth	GET /auth/github/callback	Creates or finds GitHub user, then issues tokens.
Chats	GET /api/chats	Lists current user's chats.
Chats	POST /api/chats	Creates a new chat thread.
Chats	GET /api/chats/{chat_id}	Returns one chat and persisted messages.
LLM	POST /api/chats/{chat_id}/messages	Saves user prompt, calls LLM service, saves answer.
Chats	DELETE /api/chats/{chat_id}	Deletes a chat with messages.

All /api/* routes are protected by JWT. The token can be passed as Authorization: Bearer <token> or through the HTTP-only access_token cookie used by the server-rendered UI.

2. Code structure

app/core - settings and token/password helpers.

app/db - SQLAlchemy base class, engine and session dependency.

app/models - User, Chat and Message database entities.

app/schemas - Pydantic request and response schemas.

app/controllers - FastAPI controllers for UI, auth and chat API.

app/services - business logic for auth, chats, Redis sessions, GitHub OAuth and LLM inference.

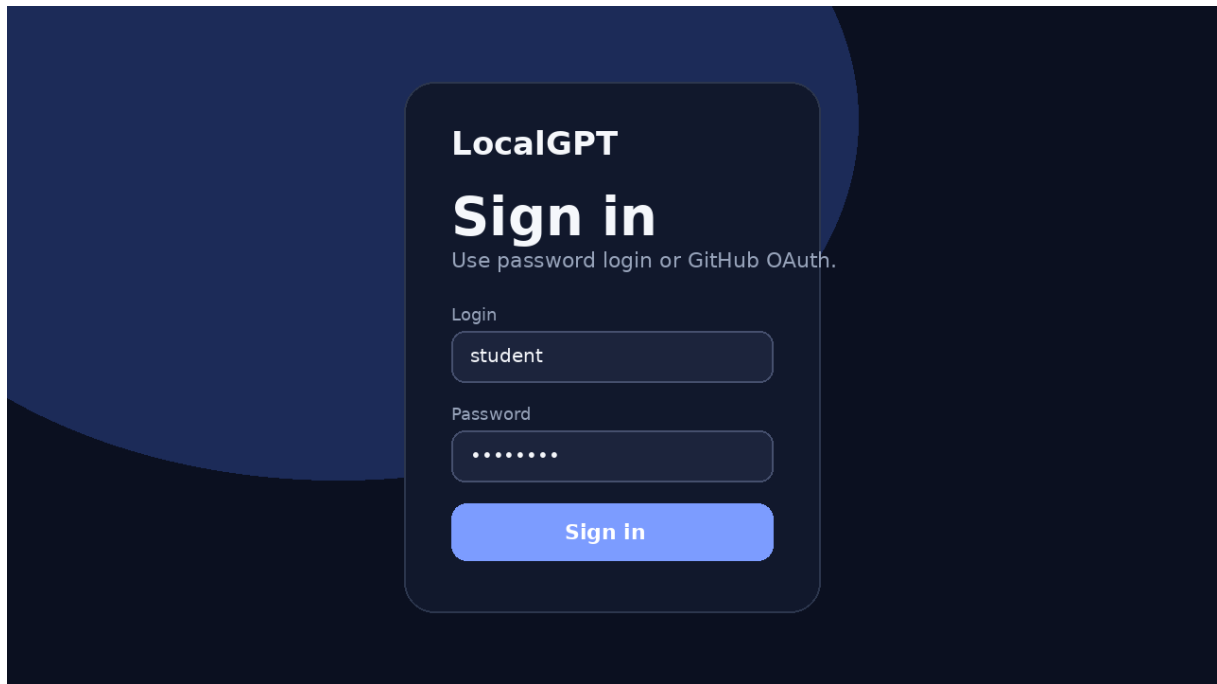
app/templates - server-rendered views.

app/static - CSS and JavaScript for chat interaction.

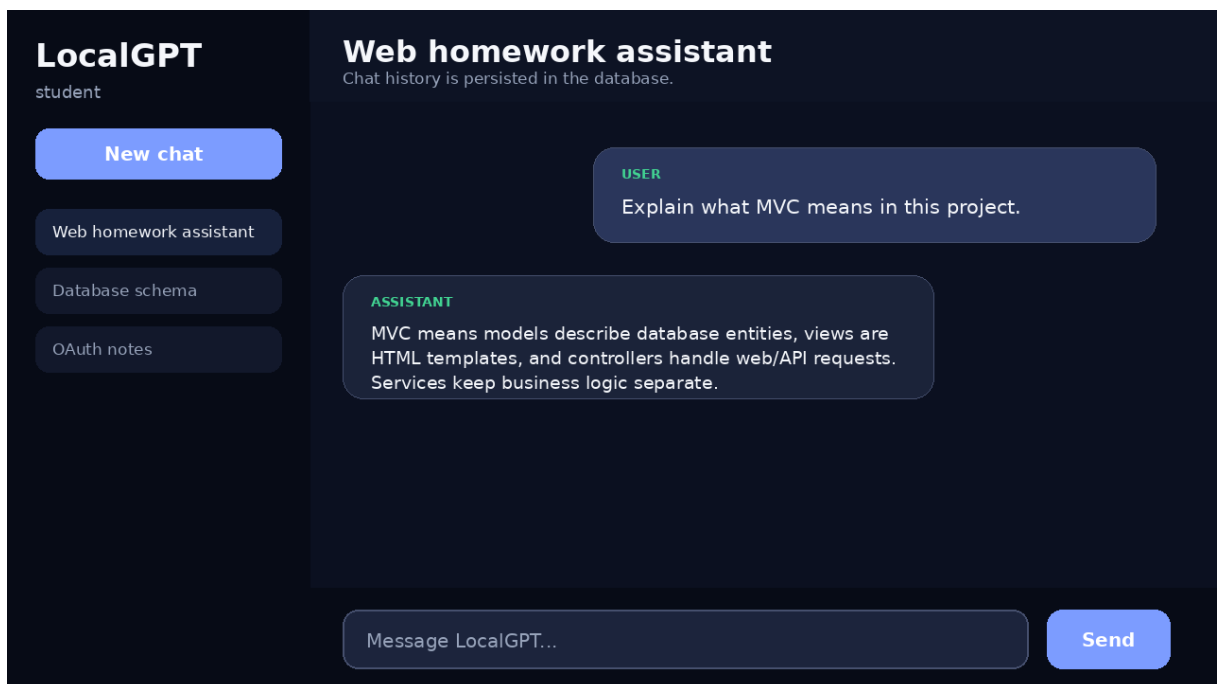
alembic - database migration environment and initial schema migration.

3. UI screenshots

Login screen



Chat screen



4. Database schema

Table	Fields	Relationships
users	id, login, password_hash, github_id, created_at	One user has many chats.
chats	id, title, owner_id, created_at, updated_at	chat.owner_id -> users.id; one chat has many messages.
messages	id, chat_id, role, content, created_at	message.chat_id -> chats.id.
Redis	refresh:{token} = user_id, login, created_at, TTL	Refresh sessions are stored for 30 days.

ERD overview

users 1 ---- * chats 1 ---- * messages

users.login is unique. Password logins store password_hash only. GitHub users store github_id and can have no password_hash.

Deleting a user cascades to chats; deleting a chat cascades to messages.

5. Design notes

- The project selects server-rendered HTML, so MVC is the required pattern.
- Controllers remain thin: they validate HTTP input and call service classes.
- Access tokens are short-lived JWTs. Refresh tokens are opaque random values stored in Redis with 30-day TTL.
- LLMService loads llama-cpp-python only when a GGUF model file exists, allowing the app to run in demo mode on a clean machine.